Segurança de Redes e Sistemas Distribuídos

2025/2026

Project – Gallery Log

1. Goal

This project is inspired in the BIBIFI security programming competition (see https://builditbreakit.org/). The aim of the project is twofold. First, it allows students to setup a secure application scenario, a *secure log*, while taking into consideration all that was learned in previous security related courses. Second, it lets students try to find real vulnerabilities in an application scenario, which was produced by other groups.

When designing a solution for your project, you **MUST create your own security protocol** (using basic security primitives, such as symmetric/asymmetric encryption, hash, MAC, signatures). You are **NOT allowed to utilize public libraries that provide complex primitives** that will significantly facilitate the design/implementation of your solution (such as an implementation of TLS if you had to design a secure channel). Your grade will be highly penalized if you do not follow these guidelines.

2. Technical Details

2.1 Overview

In this project, you will design/implement a **secure log** to describe the state of an art gallery: the guests and employees who have entered and left, and people who are in rooms. The log will be used by two programs. One program, **`logappend`**, will append new information to this file, and the other, **`logread`**, will read from the file and display the state of the art gallery according to a given query over the log. Both programs will use an authentication token, supplied as a command-line argument, to authenticate each other. Specifications for these two programs and the security model are described in more detail below.

You will **build the most secure implementation you can in the Build It phase; then you will have the opportunity to attack other teams' implementations in the Break It phase**.

To give other groups a chance to break your solution, **include a subtle vulnerability in your implementation** that might be explored in the Break It phase.

Because the other groups can't effectively start attacking your code until they can run it, you are responsible for providing a copy of your code and instructions for compiling it. **Failure to do so will penalize you in your grade!**

You will have to work in teams of **three people**. You can choose to write your implementation in either C, Java, or some other language that you may choose. You may also use any open-source libraries available online with basic cryptographic primitives.

2.2 Programs

Your team will design a log format and implement both **`logappend`** and **`logread`** programs to use it. **`logread`** includes several optional features. If you do not implement an optional feature, be sure that the program prints "***unimplemented***" to the standard output.

`logappend`

```
  logappend -T <timestamp> -K <token> (-E <employee-name> | -G <guest-
name>) (-A | -L) [-R <room-id>] <log>
  logappend -B <file>
```

Appends data to the log at the specified timestamp using the authentication token. If the log does not exist, **`logappend`** will create it. Otherwise, it will append to the existing log. If the data to be appended to the log is not consistent with the current state of the log, **`logappend`** should print "***invalid***", exit with error code 111, and leave the state of the log unchanged. The options of the program mean:

- `-T timestamp`: Time the event is recorded. This timestamp is formatted as the number of seconds since the gallery opened and is a non-negative integer (ranging from 1 to 1,073,741,823 inclusively). Time should always increase; invoking **`logappend`** with an event whose timestamp is prior to the most recent event already recorded is an error.

- `-K token`: Token used to authenticate the log. This token consists of an arbitrary-length string of alphanumeric characters (a-z, A-Z, and 0-9). Once a log is created with a specific token, any subsequent appends to that log must use the same token.

- `-E employee-name`: Name of employee. Names are alphabetic characters (a-z, A-Z) in upper and lower case. Names may not contain spaces. Names are case sensitive. Employees and guests can have the same name.

- `-G guest-name`: Name of guest. Names are alphabetic characters (a-z, A-Z) in upper and lower case. Names may not contain spaces. Names are case sensitive. Employees and guests can have the same name.

- `-A`: Specify that the current event is an arrival; can be used with `-E`, `-G`, and `-R`. This option can be used to indicate the arrival of an employee or guest at the gallery, or to a specific room with `-R`. If `-R` is not provided, `-A` indicates an arrival to the gallery as a whole. No employee or guest should enter a room without first entering the gallery. No employee or guest should enter a room without having left a previous room. Violation of either of these conditions implies inconsistency with the current log state and should result in **`logappend`** exiting with an error condition.

- `-L`: Specify that the current event is a departure, can be used with `-E`, `-G`, and `-R`. This option can be used to indicate the departure of an employee or guest from the gallery, or from a specific room with `-R`. If `-R` is not provided, `-L` indicates a departure from the gallery as a whole. No employee or guest should leave the gallery without first leaving the last room they entered. No employee or guest should leave a room without entering it. Violation of either of these conditions implies inconsistency with the current log state and should result in **`logappend`** exiting with an error condition.

- `-R room-id`: Specifies the room ID for an event. Room IDs are non-negative integer characters with no spaces (ranging from 0 to 1,073,741,823 inclusively). Leading zeros in room IDs should be dropped, such that `003`, `03`, and `3` are all equivalent room IDs. A gallery is composed of multiple rooms. A complete list of the rooms of the gallery is not available, and rooms will only be described when an employee or guest enters or leaves one. A room cannot be left by an employee or guest unless that person

has previously entered it. An employee or guest may only occupy one room at a time. If a room ID is not specified, the event is for the entire art gallery.

- `log:` The path to the file containing the event log. The log's filename may be specified with a string of alphanumeric characters (including underscores and periods). Slashes and periods may be used to reference log files in other directories. If the log does not exist, **`logappend`** should create it. **`logappend`** should append data to the log, preserving the history of the log such that queries from **`logread`** can be answered. If the log file cannot be created due to an invalid path or any other error, **`logappend`** should print "***invalid***" and return 111.

- `-B file:` Specifies a batch file of commands. `'file'` contains one or more command lines, not including the **`logappend`** command itself (just its options), separated by `\n` (newline). These commands should be processed by **`logappend`** individually, in order. This allows **`logappend`** to add data to the file without forking or re-invoking. Of course, option `-B` cannot itself appear in one of these command lines. Commands specified in a batch file include the log name. If a single line in a batch file is invalid, print the appropriate error message for that line and continue processing the rest of the batch file.

Command line arguments can appear in any order. If the same argument is provided multiple times, the last value is accepted.

After **`logappend`** finishes, the log specified by the `log` argument should be updated. The added information should be accessible to the **`logread`** tool when the token used by both programs is the same; otherwise, it should not be available (e.g., by inspecting the file directly).

Return values and error conditions:

If **`logappend`** must exit due to an error condition, or if the argument combination is incomplete or contradictory, **`logappend`** should print "***invalid***" to stdout and exit, returning a 111.

If the log to be appended already exists, and the supplied token does not match the expected token, "***invalid***" should be printed to stdout and 111 returned. (When a log file is first created, the supplied token becomes the valid token for all future accesses to that log.)

If `-B` is passed, **`logappend`** should return 0 as long as the batch file exists and valid arguments are given to the batch command. If the batch file does not exist, print "***invalid***" to stdout and exit, returning 111.

Some examples of conditions that would result in printing "***invalid***" and doing nothing to the log:

- The specified timestamp on the command line is smaller than the most recent timestamp in the existing log
- `-B` option is used in a batch file
- The name for an employee or guest, or the room ID, does not correspond to the character constraints
- Conflicting command line arguments are given, for example, both `-E` and `-G` or `-A` and `-L`

`logread`

`logread` queries the gallery's state. It prints which employees and guests are in the gallery or its rooms. The following invocations **must** be supported:

```
logread -K <token> -S <log>
logread -K <token> -R (-E <name> | -G <name>) <log>
```

The following invocation is optional (for extra points). If you do not implement the optional feature, be sure to print "***unimplemented***" to standard output when the optional argument is provided.

```
logread -K <token> -I (-E <name> | -G <name>) [(-E <name> | -G <name>)
...] <log>
```

As per the above invocations, only one of `-S`, `-R`, or `-I` may be specified at once.

In what follows, we refer to employees or visitors who are 'in the gallery'. Each person is expected to first enter the gallery (using the **`logappend`** option `-A`) before entering any particular room. Once in the gallery, he or she may enter and leave various rooms (using **`logappend`** options `-A -R` and options `-L -R`, respectively). Finally, the person will leave the gallery (using **`logappend`** option `-L`). Throughout this sequence of events, a person is considered 'in the gallery'.

When output elements are comma-separated lists, there will be no spaces before or after the commas.

- `-K token`: Token used to authenticate the log. This token consists of an arbitrary-length string of alphanumeric characters and will be the same across executions of **`logappend`** and **`logread`**. If the log cannot be authenticated with the token (i.e., it is not the same token that was used to create the file), then "***integrity violation***" should be printed to stdout and 111 should be returned.

- `-S`: Print the current state of the log to stdout. The state should be printed to stdout on at least two lines, with lines separated by the `\n` (newline) character. The first line should be a comma-separated list of employees currently in the gallery. The second line should be a comma-separated list of guests currently in the gallery. The remaining lines should provide room-by-room information indicating which guest or employee is in which room. Each line should begin with a room ID, printed as a decimal integer, followed by a colon, followed by a space, followed by a comma-separated list of guests and employees. Room IDs should be printed in ascending integer order, and all guest/employee names should be printed in ascending lexicographic string order.

- `-R`: Give a list of all rooms entered by an employee or guest. Output the list of rooms in chronological order. If this argument is specified, either -E or -G **must** be specified. The list is printed to stdout as a single comma-separated list of room identifiers. This list should include all rooms visited over the history of the log, regardless of how many separate visits the employee/guest has made to the gallery. If the specified employee or guest does not appear in the gallery log, then nothing is printed.

- `-I`: Prints the rooms, as a comma-separated list of room IDs, that were occupied by all the specified employees and guests at the same time over the complete history of the gallery. Room IDs should be printed in ascending numerical order. If a specified employee or guest does not appear in the gallery, it is ignored. If no room ever contained all of the specified persons, then nothing is printed. This feature is optional.

- `-E employee_name`: May be specified multiple times when used with `-I`.

- `-G guest_name`: May be specified multiple times when used with `-I`.

- `log`: The path to the file log used for recording events. The filename may be specified with a string of alphanumeric characters (including underscores and periods). Slashes and periods may be used to reference log files in other directories.

Command line arguments can appear in any order. If the same argument is provided multiple times, it should be considered an error. If **`logread`** is invoked with an incomplete, contradictory, or otherwise non-compliant command line, it should print "***invalid***" to stdout and exit, returning 111.

If **`logread`** is given an employee or guest name that does not exist in the log, it should print nothing about that employee/guest (which may result in empty output).

If **logread** cannot validate that an entry in the log was created with an invocation of **logappend** using a matching token, then **logread** should return a 111 and print "***integrity violation***" to stdout.

Example return values and error conditions:

- `-I` used specifying an employee that does not exist in the log, the program should print nothing and exit with return 0.
- If the log file has been corrupted, the program should print "***integrity violation***" to stdout and exit with return 111.

2.3 Examples

Consider the following 4 invocations of **logappend**:

```
$ ./logappend -T 1 -K secret -A -E Fred log1
$ ./logappend -T 2 -K secret -A -G Jill log1
$ ./logappend -T 3 -K secret -A -E Fred -R 1 log1
$ ./logappend -T 4 -K secret -A -G Jill -R 1 log1
```

These commands used the key secret to append 4 events to `log1`, recording the arrival of `Fred` and `Jill` in room `1` of the gallery. If **logread** is then used to print the state of the gallery, the following should be printed:

```
$ ./logread -K secret -S log1
Fred
Jill
1: Fred,Jill
```

If we continue using `log1` and record some movements, we can then use **logread** to get a list of the rooms Fred entered.

```
$ ./logappend -T 5 -K secret -L -E Fred -R 1 log1
$ ./logappend -T 6 -K secret -A -E Fred -R 2 log1
$ ./logappend -T 7 -K secret -L -E Fred -R 2 log1
$ ./logappend -T 8 -K secret -A -E Fred -R 3 log1
$ ./logappend -T 9 -K secret -L -E Fred -R 3 log1
$ ./logappend -T 10 -K secret -A -E Fred -R 1 log1
$ ./logread -K secret -R -E Fred log1
1,2,3,1
```

We can also use **logappend** in batch mode like so (on a fresh log `log2`):

```
$ cat batch
  -K secret -T 0 -A -E John log2
  -K secret -T 1 -A -R 0 -E John log2
  -K secret -T 2 -A -G James log2
  -K secret -T 3 -A -R 0 -G James log2
$ ./logappend -B batch
  invalid
  invalid
$ echo $?
  0
$ ./logread -K secret -S log2
```

```
  James
  0:James
$ echo $?
  0
```

The first two lines of the batch file are invalid: since 0 is an invalid time so the first line fails; then the second line fails with "*invalid*" since John is not in the gallery. Here is a successful run with the time arguments fixed:

```
$ cat batch
  -K secret -T 1 -A -E John log2
  -K secret -T 2 -A -R 0 -E John log2
  -K secret -T 3 -A -G James log2
  -K secret -T 4 -A -R 0 -G James log2
$ ./logappend -B batch
$ echo $?
  0
$ ./logread -K secret -S log2
  John
  James
  0:James,John
$ echo $?
  0
```

2.4 Security Model

The system as a whole must guarantee the confidentiality and integrity of the log in the presence of an adversary that does not know the authentication token. This token is used by both the **`logappend`** and **`logread`** tools, specified on the command line. Without knowledge of the token, an attacker **should not** be able to:

- Query the logs via **`logread`** or otherwise learn facts about the names of guests, employees, room numbers, or times by inspecting the log itself
- Modify the log via **`logappend`**.
- Fool **`logread`** or **`logappend`** into accepting a bogus file. In particular, modifications made to the log by means other than correct use of **`logappend`** should be detected by (subsequent calls to) **`logread`** or **`logappend`** when the correct token is supplied.

The tools could also be targets of attacks that cause abnormal behavior during execution. In this case, the attacker should not be able to **crash** the tools.

`Crash Violations`

A crash occurs when a program unexpectedly terminates, for example, due to a memory-safety violation. To demonstrate a crash, provide a testcase (with your report) with the
`commands` and attributes necessary to demonstrate the crash.
For crash violations, provide a textual justification for why you think the testcase you are submitting could potentially be used to create security problems by exploiting a memory violation (or some other problem). Do this in your report. It may refer to relevant portions of the line numbers in the submitted code, if you like.

`Integrity Violations`

An integrity violation occurs when an attacker successfully modifies a log without knowledge of the token used to produce it. Evidence of successful modification is that the modified file is correctly interpreted by **`logread`** (when using the token used to create the original log). The tokens for the supplied log files will not be provided to students.

To demonstrate that this is possible, submit a testcase where you: First, create the log file using correct commands and attributes (including the token). Alternatively, you can use one of the three files provided by the team that developed the software; Second, you explain how you modify the log file without accessing the token; Third, you explain how the modified log file would be processed correctly by **`logread`** (with the correct token).

A correctly implemented program should always exit with the output "integrity violation"; an incorrectly implemented program will produce some other output.

`Confidentiality Violations`

A confidentiality violation occurs when an attacker can infer information about the contents of a log without knowledge of the token. Evidence of such a violation is a successful guess of an outcome of a query on the file, despite not knowing the token (or the transcript of the commands used to create it).

To demonstrate that this is possible, you can produce a testcase with the format: First, create the log file using correct commands and attributes (including the token). Alternatively, you can use one of the three files that are given by the team that developed the software; Second, you explain (including the commands) how you can infer information about the contents of the log; Third, when we run your testcase, we will execute with the token, and thus if the expected output is indeed produced, it is clear that the break-it team has reverse-engineered the contents of the file.

NOTE: If you managed to perform an attack that cannot be described with one of the above testcase formats, then you need to explain the attack in detail in your report.

2.5 Deliverables

The professor will evaluate your work based on the information you provide and the group discussion that occurs at the end of the semester. In each phase, you will deliver both a document and some implementation code (see below). **Most of the analysis of your work will be based on the documents provided, and the code will be used only to clarify specific aspects of the solution**. Therefore, **ensure that the documents are detailed and explain your ideas clearly** (otherwise, your grade might be unnecessarily penalized).

Part 1: Build it

You should submit:

- A **design document** (PDF) in which you:
  - Describe your overall system design in sufficient detail for a reader to understand your approach **without reading the source code directly**. This must include a description of the format of your log file and the security protocol that you have designed.
  - Discuss the performance of your solution. **Solutions with better performance will receive a higher grade.**
  - List four specific attacks you have considered and describe how your implementation counters these threats (Please indicate the relevant lines of code that implement your security controls). If you were not able to completely (or at all) prevent a threat you identified, you may still mention it. In that case, describe any partial mitigation you implemented and explain anything you wanted to implement but

were, for whatever reason, unable to. Please be clear about distinguishing what you have implemented vs. what you *would have* implemented.
  - A description of the *subtle vulnerability* that you have introduced in your code. Discuss also why you picked this vulnerability and why you consider it "subtle".

- Your **implementation**, including all your code files and your makefile. You should also provide **three log files**, containing the result of three executions of different command sets. You **must not** provide the used tokens to the other student teams (they **should only** be provided to the professor by writing them in the design document)!!!!!

Part 2: Break it

We will randomly assign you **two** teams' implementations to examine. You should submit:

- A **vulnerability analysis document** (PDF). For each of your assigned implementations and describe:
  - Any attacks you found, including a **high-level summary** and **enough detail for someone to replicate** the attack.
  - Any vulnerabilities you found but **were unable** to exploit for whatever reason.
  - If you did **not** find any attacks or vulnerabilities, describe **how you looked for attacks**.

- Also submit **any code you wrote to implement your attack**. Make sure the vulnerability analysis explains how to use your code to launch the attack. Also, include the **testcases that demonstrate** a successful attack. Attacks can include crashes, integrity violations, and confidentiality violations.

Please note that while correctness bugs are relevant, they do not count for your vulnerability analysis -- for this analysis, you **must identify security-relevant issues**.

If one of your assigned implementations does not work well enough to analyze, please request an alternate implementation from the professor.

3. Timeline

Each group should finish the first phase of the project by the **13th of April at midnight**. An electronic copy of the report should be sent **before** that time to nuno@di.fc.ul.pt, and a copy of the report and implementation should be saved in Moodle.

The second phase should be completed by the **18th of May at midnight**. Again, an electronic copy of the report should be sent **before** that time to nuno@di.fc.ul.pt, and a copy of the report and other deliverables should be saved in Moodle.

No extensions will be provided.

4. Grading

Recall that most of your evaluation will be based **on the documents** you deliver. The code will be run or inspected in exceptional cases. Students will be evaluated by the following criteria:

Part 1: Build it

- your design document:

- the description of your approach
- each of the four specific vulnerabilities you defend against. For maximum points, your defense should be correct, fully implemented, and well-explained. Well-explained attacks that are not fully implemented will receive partial credit.
- the vulnerability that was inserted.

Part 2: Break it

- A successful attack with a well-written vulnerability analysis will receive more credits.

- If you do not have a successful attack, then you must write a complete vulnerability analysis for both implementations you were assigned. This analysis must explain either why you could not implement the attack you identified or provide a convincing argument that there were no exploitable vulnerabilities.